1. What If There Were No Operating System?

If an operating system does all the things we know it does, it seems downright impossible for a computer to exist without one.

In reality, the earliest computers didn't have operating systems; they were huge machines tasked with one program at a time. For that reason, they didn't really need operating systems. In fact, the earliest computers required a user to physically connect and disconnect wires from a plug board to retrieve computations. But if you don't have an operating system, can you make your computer do anything?

Yes. But you have a lot of work to do. Without an operating system using and enforcing a standard, systematic approach to running the computer, you're put in the position of writing code that must tell the computer exactly what to do. So, if you want to type up a document in a word processing program, you'd have to create from scratch code that tells your computer to respond to each character pressed on your keyboard. Then you'd have to write a code that told the computer how those responses must translate to a screen. You'd have to tell your computer how to draw the character you want. Think of every single option or possibility your word processing program has. You'd have to write code for every single one of those directly onto your hard drive.

In the absence of an OS, your PC will boot using a small piece of firmware known as the BIOS (Basic Input/Output System). The BIOS governs very simple features such as resetting the clock, voltage regulation or diagnosing system errors. Its most useful function is the ability to select an installed disk from which to boot the proper OS, so it's not going to be able to handle complex tasks like word processing or web browsing.

An operating system provides a uniform set of screws, lumber and any other material you need. It can go back and forth between rooms so fast you didn't even know it left the one you were in.

And that's really important, because here's another thing: Remember how we were talking about the operating system only being able to concentrate on one thing at a time? Well, without one, your computer could run one program. Period. You could create a document. You could save it. You could

print it. But you couldn't look at that document and keep a clock running on your desktop. If you don't have an operating system, you're stuck doing one and only one process at a time.

To conclude , a computer is down to bare hardware with no OS, and you basically have an expensive calculator that can only run exactly what you personally load into it in machine code.

Problems without an OS:

- Only one program could ever run at a time — no scheduler means the CPU just does what it's told until it's done, no switching between apps.
- No file system — data on disk is just raw bytes. No "files" or "folders," so every program has to track its own disk addresses, with nothing stopping it from overwriting someone else's data.
- No hardware abstraction — every app would need its own driver for the exact keyboard, disk, graphics card in use. A generic word processor couldn't exist.
- No security or accounts — anything running could read, modify, or delete anything else.
- No memory protection — two programs could get overlapping memory addresses and silently corrupt each other.

How the OS fixes each:

- **Process management** — the kernel's scheduler gives every process a CPU time-slice and switches fast enough it looks simultaneous.
- **File systems** (ext4, NTFS...) — organize raw disk space into files/folders and track where things live.
- **Device drivers** — standard interface so apps just say "print this" without knowing the hardware.
- **Users/permissions** — enforce who can read, write, execute what.
- **Memory management** — hands each process its own virtual address space, no collisions.

Demonstrating 3 features on Linux

1. Process management — running a background job and checking what the OS is juggling

I started a background process and then listed what the scheduler currently has in its queue:

```
$ sleep 30 &
[1] Running                 (PID 487)
$ ps aux | head -5
USER   PID  %CPU   %MEM  VSZ      RSS    TTY  STAT START  TIME COMMAND
root     1      4.8         0.1        2860  5560  ?        SLl  14:55  0:01 /process_api ...
root     2   0.0  0.0         0     0  ?S    14:55  0:00 [kthreadd]
```

The system is already running dozens of kernel and background processes even before I started anything, and my sleep job got picked up and scheduled alongside them with its own PID — that's the process manager at work, doing exactly what couldn't happen without an OS.

2. Memory management — checking how RAM is being tracked and shared

```
$ free -h
          total   used   free   shared  buff/cache  available
Mem:              3.9Gi  209Mi  3.8Gi  4.8Mi   74Mi        3.7Gi
Swap:             0B        0B     0B
```

The OS is keeping a live account of how much memory is in use, free, cached, or shared across every running process. Without this layer, every program would have to guess which physical addresses were safe to use.

3. File system and permissions — creating a file and locking it down

```
$ touch demo.txt
$ ls -l demo.txt
-rw-r--r-- 1 root root 0 Aug 14 14:56 demo.txt
$ chmod 600 demo.txt
$ ls -l demo.txt
-rw------- 1 root root 0 Aug 14 14:56 demo.txt
```

The file starts out readable by everyone (rw-r--r--). After chmod 600, only the owner can read or write it — nobody else, not even other logged-in users. That's the OS enforcing access control at the file-system level, which simply has no equivalent on a machine with no OS.

2. Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or "OS selection guide."

You can't run one distro for everything in a real lab- different jobs want different tools.

1. Kali Linux — Penetration Testing. Debian-based, maintained by Offensive Security, ships with Nmap, Metasploit, Burp Suite, Wireshark, John the Ripper, etc. Industry standard, most certs (OSCP) built around it. Best for: authorized pentests and red-team work.

2. Parrot OS (Security Edition) — Penetration Testing / Lightweight. Similar toolset to Kali, also Debian-based, but lighter on resources with built-in anonymity tooling (AnonSurf, routes traffic through Tor). Best for: pentesting on modest hardware, or testers wanting anonymity too.

3. CAINE — Digital Forensics. Built for forensic integrity — mounts drives read-only by default so evidence can't be altered. Comes with Autopsy, The Sleuth Kit. Best for: incident response and forensic investigation after a breach.

4. Tails — Privacy. Boots from USB, runs in RAM, forces all traffic through Tor, leaves zero trace on the host machine when shut down. Best for: anonymous research/communication where leaving no forensic trace matters.

5. Security Onion — Security Monitoring / Blue Team. Flips to defense — network security monitoring with Suricata and Zeek for intrusion detection, Elastic Stack for logs/dashboards. Best for: continuous monitoring and blue-team work.

Selection guide

Role	Distribution	Why
Penetration testing	Kali Linux	Industry-standard tools, cert-aligned.
Pentesting	Parrot Security	Same tools, lighter, built-in anonymity
Digital forensics	CAINE	Read-only evidence handling
Privacy / anonymity	Tails	Amnesic live boot, all traffic via Tor
Security monitoring	Security Onion	Built-in IDS + log/dashboard stack.

A real lab would likely run several together: Kali/Parrot on the attack box, Security Onion watching the network, CAINE on standby for investigations, Tails on a USB for anyone needing anonymity outside the lab.

In Linux, an exit status of **0 generally means that the command was successful**, while a **non-zero value indicates that something went wrong or the command was terminated in some way**.

Commands Executed and Observations

Command	Result	Exit Status	Meaning
whoami	Displayed the current username msis	0	The command executed successfully.

ls -l	Displayed the detailed list of files and directories	0	The command completed successfully.
help help	Displayed information about the Bash help command	0	The command was executed successfully.
ls Abhivarsha	Displayed “No such file or directory”	2	The requested file/directory was not found.
sleep 11	The command was interrupted using Ctrl+C	130	The command was terminated by the interrupt signal.
chmode +x	Linux reported that chmode was not found	127	The command could not be found.
ls abcde	Displayed “No such file or directory”	2	The specified file/directory did not exist.
when	Linux reported that the command was not found	127	The shell could not find the command.

These results are taken from my actual terminal screenshots. For example, whoami and ls -l returned 0, while ls Abhivarsha returned 2. The sleep 11 command was interrupted and returned 130, and the invalid commands chmode +x and when returned 127.

What Can a Script Learn from an Exit Status?

An exit status allows a shell script to know whether the previous operation worked or failed. The script can then make a decision based on the result instead of blindly continuing.

For example:

ls test.txt

```
if [ $? -eq 0 ]; then
    echo "File found"
else
    echo "File not found"
fi
```

If ls succeeds, the exit status is 0, so the script prints **“File found.”** If the command fails, the status is non-zero and the script can handle the error.

This is useful in real scripts because one operation may depend on another. For example, a script could check whether a file exists before trying to copy it, check whether a directory was created before using it, or stop a process when an important command fails.